Ain Shams University
Faculty of Engineering
Mechatronics Engineering Department

MCT331s · Mechatronic System Integration · Fall 2026

Smart Home Prototype Progress Report 1

Home structure and window security system

Team member	ID
Mahmoud Hisham	2400844
Ziad Hany	2301401
Ahmed Hany	2400311
Moatasem Mohamed	2400732

Progress session: 21–22 October 2026

Contents

1. Summary
2. The project in short
3. Home structure
4. Security system (window alarm)
5. Problems we faced and how we solved them
6. Next steps
7. Appendix A: full code · Appendix B: planned pin map · Appendix C: project files

1. Summary

For this progress session we were asked to show two things: **(1)** the home structure, built as a real engineering prototype, and **(2)** a working window alarm that sends an alert when the window is opened.

Progress 1 item	Status	What we did
Home structure (layout, material, build)	**Done**	Designed in CAD, laser-cut from 3 mm MDF and assembled with finger joints and glue. 300 × 220 × 150 mm house on a 320 × 240 mm base with a removable flat roof.
Window intrusion detection	**Done**	An MC-38 magnetic sensor on the window. When the window opens while security is on, the buzzer sounds and the LCD shows `!! INTRUSION !!`. When the window closes, the buzzer stops.
LCD feedback	**Done**	A 16×2 I2C LCD shows the security mode and the window state at all times.
Alert to the designated recipient(s)	**In progress**	The Arduino already sends the message `ALERT:WINDOW_OPEN` over serial on every new intrusion. The next step is an ESP8266 Wi-Fi board that forwards it to a phone.

2. The project in short

We are building a small smart home controlled by an **Arduino UNO**. The final house will have five systems. This report covers the parts asked for in Progress 1: the **house**, the **window alarm** and the **LCD**.

System	What it does	Main parts	Status
Main door access	Opens the door when the right code is typed, then closes it after a few seconds	4×4 keypad, SG90 servo	Next
Security system	Alarm if the window opens; outside light turns on if someone moves near the door	MC-38 sensor, buzzer, PIR, LED, toggle switch	**Window part done**
Indoor lighting	Light turns on when it gets dark, or with a button	LDR, 3 LEDs, push button	Next
Climate control	Fan turns on when the room is hotter than the value set by a knob	DHT11, DC motor + propeller, potentiometer	Next
LCD display	Shows the state of every system	16×2 LCD with I2C module	**Working**

3. Home structure

3.1 Material and why we chose it

The house is made of **3 mm MDF** (wood fibre board), cut with a **laser cutter** from drawings we made in CAD.

- **Accurate:** the laser cuts every hole to within a fraction of a millimetre, so the sensors, LCD and buttons fit exactly.
- **Strong enough and light:** 3 mm MDF is stiff for a house this size and easy to glue.
- **Cheap and fast:** the whole house comes out of **two 600 × 400 mm sheets**.
- **Easy to work with:** it can be drilled, sanded or painted later if needed.

3.2 Main dimensions

Item	Size (mm)
House footprint (outside)	`300 × 220`
Wall height	`150`
Total height with lid and knob	`159`
Base plate	`320 × 240`
Roof lid (removable)	`306 × 226`
Main door opening	`60 × 100`
Window opening	`70 × 55`
Fan opening	`Ø 62`
Electronics bay (behind the inner wall)	`294 × 75`

3.3 How the parts hold together

- **Finger joints at every corner:** the walls have teeth (7 per corner, 21.4 mm each) that lock into each other like puzzle pieces. This keeps the box square.
- **Tabs and slots in the base:** every wall has small tabs on its bottom edge that drop into slots in the base, so each wall can only go in one place.
- **Wood glue (PVA)** on all joints. No screws are needed for the shell.
- **Removable lid:** four small L-shaped pieces under the lid sit inside the walls and stop it from sliding. Lifting the lid gives full access to the wiring.

3.4 Inside layout: where everything goes

An inner wall (partition) splits the house into two zones. The front zone is the **living room** with all the sensors and outputs. The back zone (75 mm deep) is the **electronics bay**, where the Arduino and the breadboard sit hidden. The Arduino USB port lines up with a hole in the back wall, so we can upload code without opening the house.

Where	What is mounted there
Front wall	Main door (servo above it), keypad, LCD, PIR motion sensor, outside light, ARM switch, buttons, fan knob
Right wall	**Window with the MC-38 sensor**, light sensor (LDR)
Left wall	Exhaust fan (motor + propeller in a small duct)
Inner wall	Indoor lights (3 LEDs), temperature sensor (DHT11)
Electronics bay	Arduino UNO on standoffs, breadboard, buzzer, wiring
Back wall	USB and power holes for the Arduino, air vents

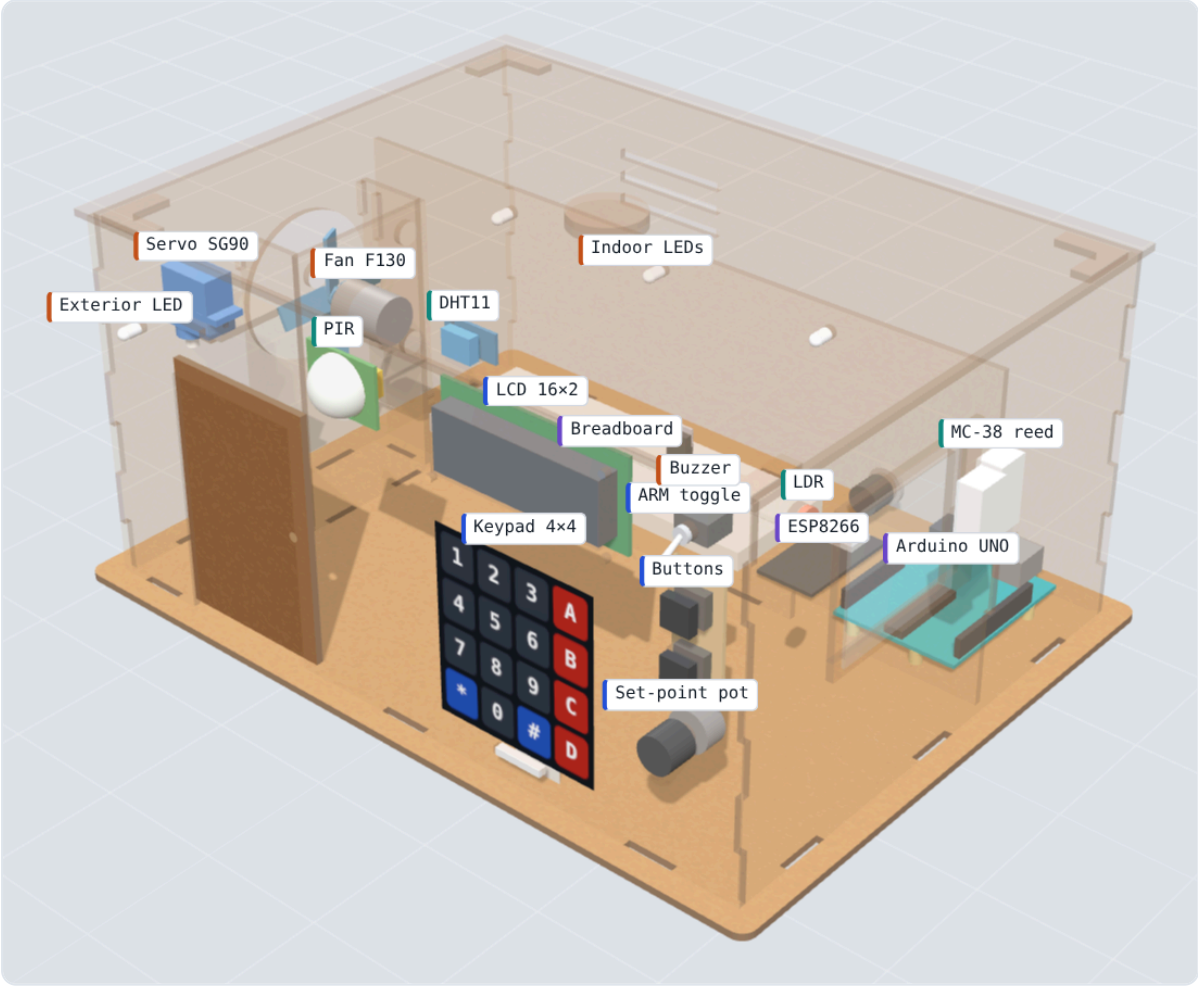


Figure 1. CAD model with see-through walls showing where each component sits.

3.5 CAD views

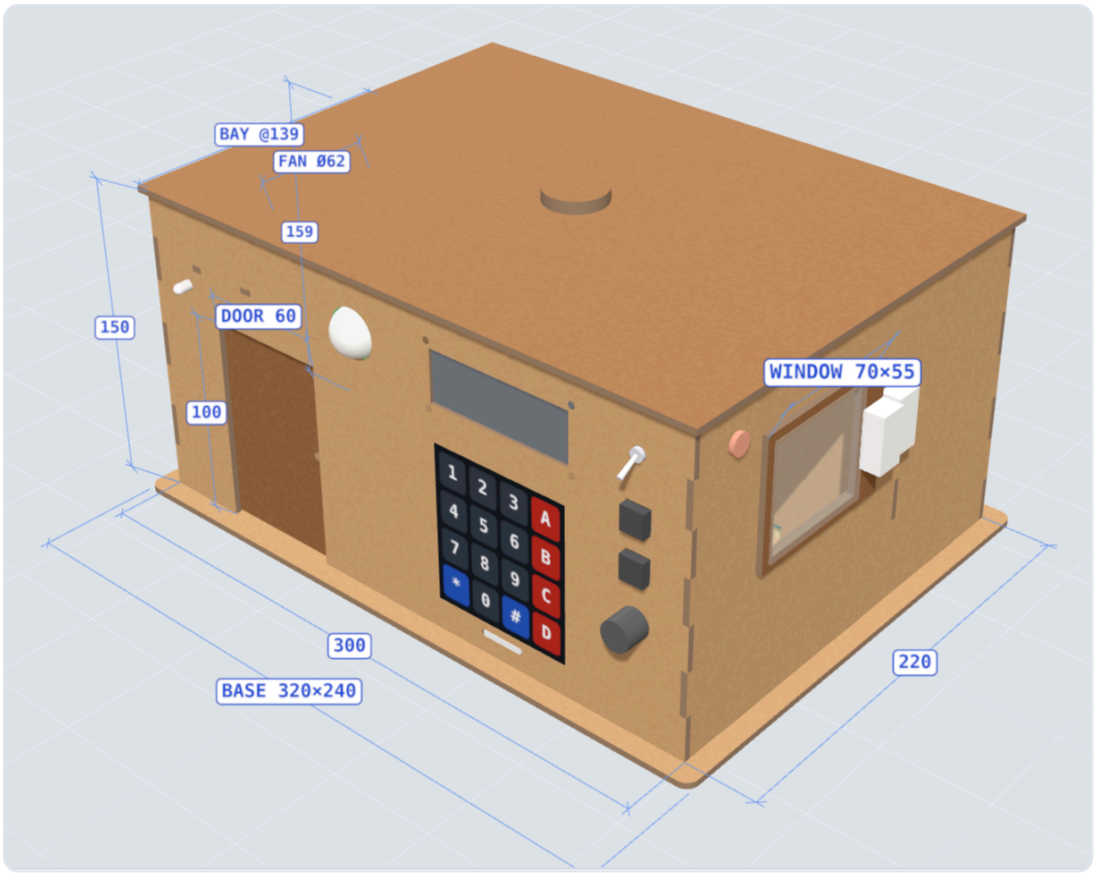


Figure 2. Isometric view with the main dimensions (mm).

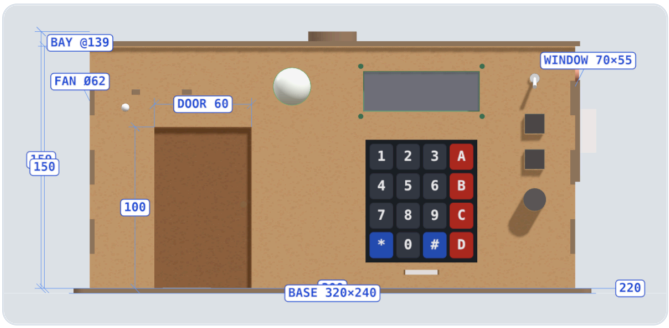


Figure 3. Front view: door, keypad, LCD, PIR and controls.

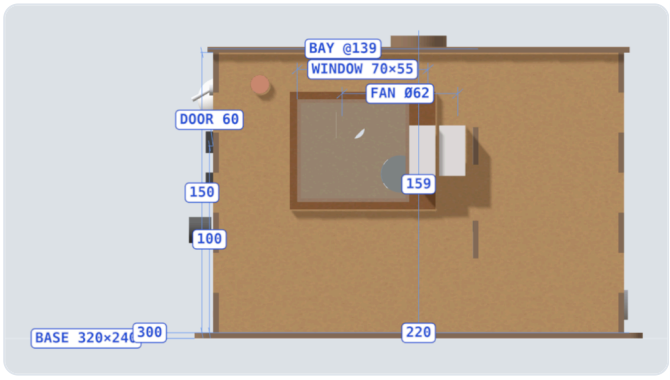


Figure 4. Right view: window with the MC-38 sensor.

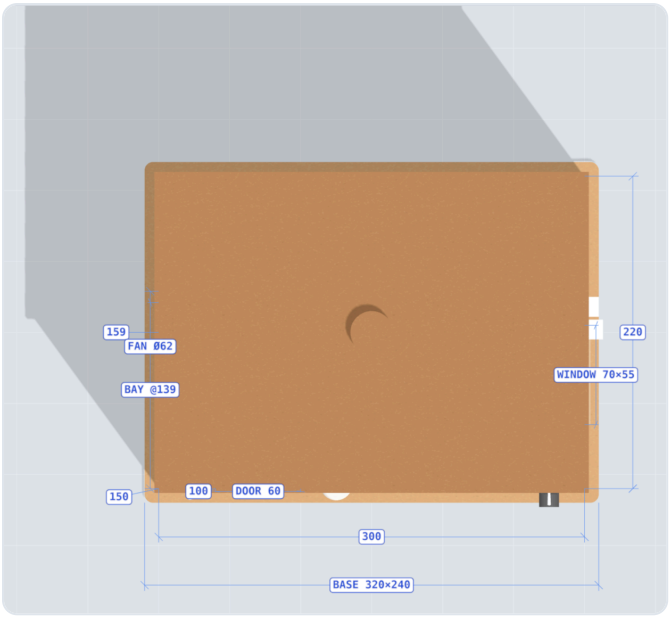


Figure 5. Top view (with the lid on).

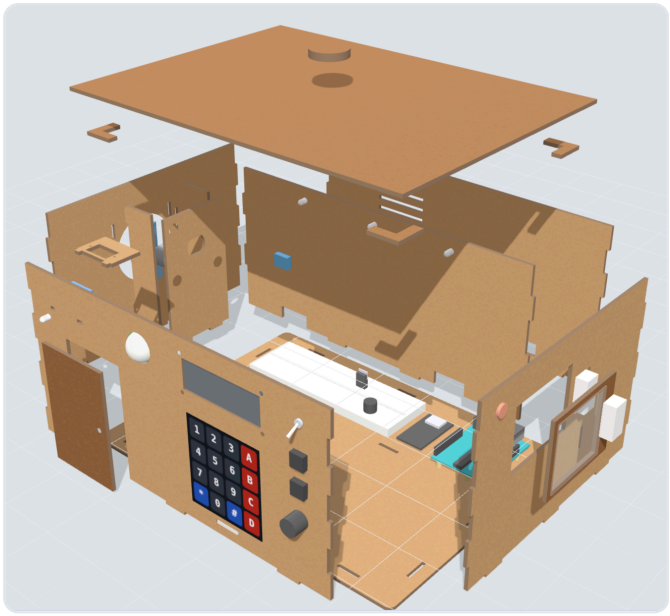


Figure 6. Exploded view: how the panels fit together.

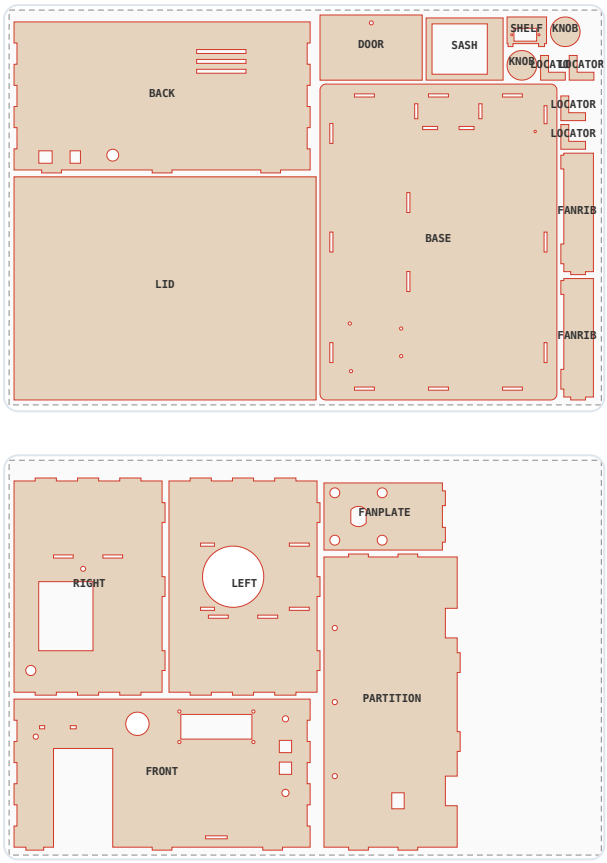
3.6 Laser-cut parts

The design has **19 parts** (14 different shapes), nested on **2 sheets of 600 × 400 mm**. Total cut length is **12.5 m**. The team names are engraved under the base.

Part	Flat size (mm)	Qty	What it does
Front wall	`300.0 × 153.0`	1	Main door, keypad, LCD, PIR, exterior light, control buttons
Back wall	`300.0 × 153.0`	1	USB and power access for the Arduino, air vents
Left wall (fan)	`220.0 × 153.0`	1	Exhaust fan opening (Ø62 mm)
Right wall (window)	`220.0 × 153.0`	1	Window opening (70 × 55 mm), light sensor, window sensor
Partition (bay wall)	`300.0 × 138.0`	1	Separates the living room from the electronics bay
Fan motor plate	`68.0 × 123.0`	1	Holds the fan motor
Fan duct rib	`33.0 × 123.0`	2	Sides of the fan duct
Servo shelf	`40.0 × 30.0`	1	Holds the door servo above the door
Main door	`66.0 × 103.5`	1	Main door (moved by the servo)
Window sash (frame)	`78.0 × 63.0`	1	Window frame (opens outward on a hinge)
Base plate	`320.0 × 240.0`	1	Floor; all walls lock into its slots
Flat roof lid	`306.0 × 226.0`	1	Removable flat roof
Lid corner locator	`25.0 × 25.0`	4	Keep the lid in place on the walls
Lid knob disc	`30.0 × 30.0`	2	Handle to lift the lid
Total		**19**	**All from 3 mm MDF**

3.7 Cutting layout

These are the two sheets sent to the laser cutter. Red lines are cut through, and the blue lines are light engraving (labels).



> **Saving material:** our first design needed 3 sheets. We lowered the walls from 160 to 150 mm and replaced a full frame under the lid with four small corner pieces. The same house now fits on 2 sheets (**33 % less MDF**), with 11 % less cutting.

4. Security system (window alarm)

4.1 What it does

- When **security mode is ON** and someone **opens the window**, the buzzer sounds **continuously** and the LCD shows `!! INTRUSION !!`.
- When the window is **closed again**, the buzzer **stops right away** and the LCD shows `Window: CLOSED`.
- At the moment the alarm starts, the Arduino sends the message `ALERT:WINDOW_OPEN` **once**. This message is what will be forwarded to the phone.
- Security mode is turned on and off with the **ARM toggle switch** (pin D7). In our bench tests the code setting `ALWAYS_ARMED = true` kept security on without the switch. With the switch connected, this setting becomes `false`.

4.2 Components used

Component	Class	Job in this system	Arduino pin
Arduino UNO R3	Information processing	Reads the sensor, decides, drives the outputs	-
MC-38 magnetic window sensor	Sensor	Tells whether the window is open or closed	D2
Buzzer 5 V	Actuator	Alarm sound	D8
LCD 16×2 with I2C module	Information processing (display)	Shows the security mode and window state	A4 (SDA), A5 (SCL)
Toggle switch	Sensor (user input)	Turns security mode on and off	D7
Breadboard 830 + jumper wires	Mechanical / wiring	Shares 5 V and GND, joins the parts	5V, GND

4.3 How the window sensor works

The MC-38 has two pieces: a **switch** fixed on the wall (with two wires) and a **magnet** fixed on the window. Inside the switch is a tiny metal contact that closes when the magnet is near.

We connect one wire to **D2** and the other to **GND**, and turn on the Arduino's built-in pull-up resistor (`INPUT_PULLUP`). The pull-up makes the pin read HIGH when nothing pulls it down, so no extra resistor is needed.

Window	Magnet	Switch inside	D2 reads	Meaning in the code
Closed	Next to the switch	Closed (joins D2 to GND)	LOW	`windowOpen = false`
Open	Away from the switch	Open	HIGH	`windowOpen = true`

> The magnet must be within about **10 mm** of the switch when the window is closed. We mounted them side by side on the window edge.

4.4 Wiring

The Arduino's 5 V and GND go to the breadboard power rails first, and every part takes power from there. The numbers on the wires match the table below.

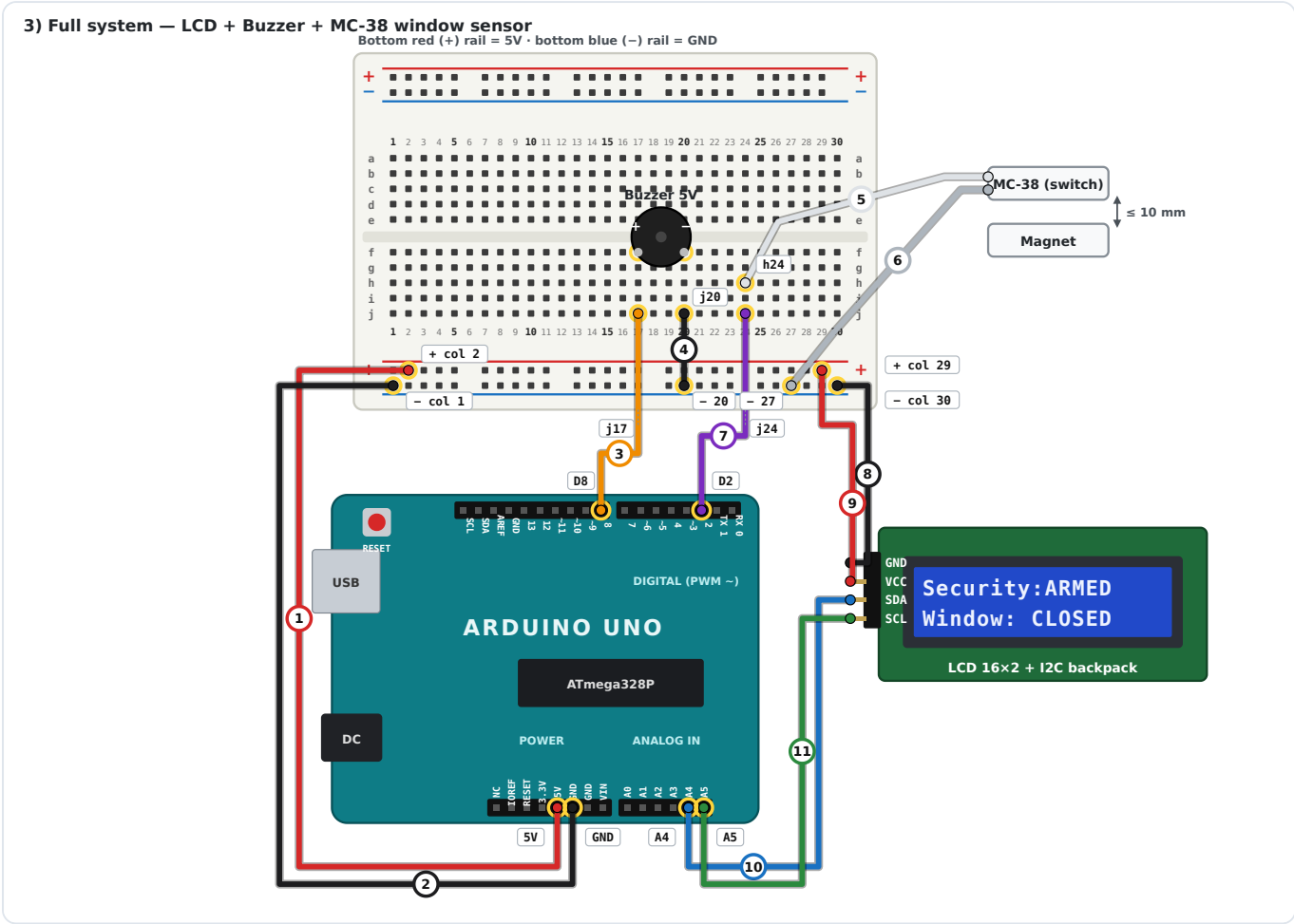


Figure 7. Wiring of the LCD, buzzer and window sensor (breadboard view).

#	From	To	Wire
1	Arduino 5V	Rail + (col 2)	Red
2	Arduino GND	Rail − (col 1)	Black
3	Arduino D8	j17 (buzzer + leg in f17)	Orange
4	j20 (buzzer − leg in f20)	Rail − (col 20)	Black
5	MC-38 wire 1	h24	Sensor wire
6	MC-38 wire 2	Rail − (col 27)	Sensor wire
7	Arduino D2	j24	Purple
8 / 9	LCD GND / VCC	Rail − (col 30) / Rail + (col 29)	Black / Red
10 / 11	LCD SDA / SCL	Arduino A4 / A5	Blue / Green

4.5 Control logic (flowchart)

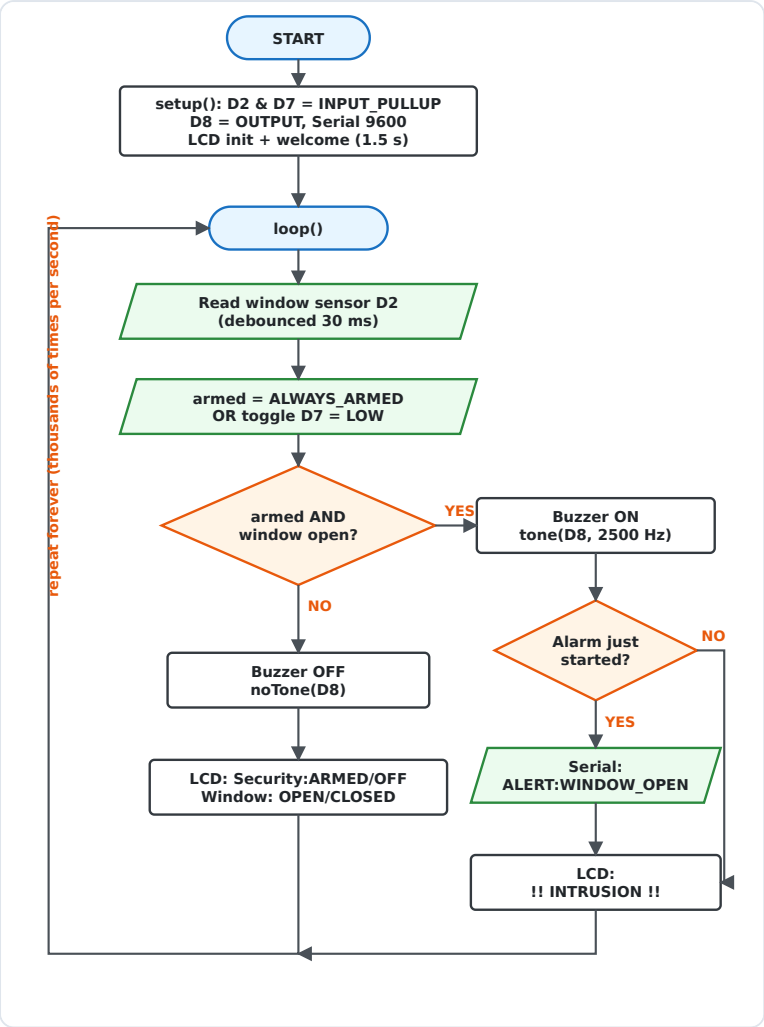


Figure 8. Flowchart of the security program.

In simple words:

1. When the Arduino turns on, `setup()` runs **once**. It sets up the pins, starts the LCD and shows a welcome message.
2. Then `loop()` runs **again and again**, thousands of times every second.
3. Each time, it reads the window sensor and the ARM switch.
4. If security is ON **and** the window is open, the buzzer turns on and the LCD shows the alarm. The first time this happens, it also sends `ALERT:WINDOW_OPEN`.
5. Otherwise, the buzzer is off and the LCD shows the normal status.

> The program never uses `delay()` inside the loop, so it always reacts at once. Only the 1.5 s welcome message in `setup()` waits.

4.6 The main ideas in the code

Part of the code	What it does, in simple words
`readDebounced()`	A metal switch "shakes" for a few milliseconds when it opens or closes, and the Arduino can see many fake changes. This function only accepts a new reading after it stays the same for **30 ms**. That keeps the alarm from flickering.
`buzzer(on)`	Turns the buzzer on with `tone(D8, 2500 Hz)` or off with `noTone()`. It only acts when the state **changes**, so the sound stays smooth. `tone()` works for both buzzer types (active and passive).
`lcdWrite()`	Writes the two LCD lines, padded to 16 characters so old letters are erased. It only rewrites a line when its text changes, so the screen does not flicker.
`updateSecurity()`	Reads the sensor and the switch and sets `alarmOn = armed AND windowOpen`. Then it drives the buzzer and sends the alert message once when the alarm starts.
`updateLCD()`	Shows `!! INTRUSION !!` during an alarm. Otherwise it shows `Security:ARMED/OFF` and `Window: OPEN/CLOSED`.
`Settings at the top`	All pin numbers and options are at the top of the file (LCD address `0x27`, sensor `D2`, buzzer `D8`, switch `D7`), so they are easy to change.

The full code is in Appendix A. Every function has a comment that explains its job, inputs and output.

4.7 Tests and results

#	Test	Expected	Result
1	Power on	LCD shows `SMART HOME / MCT331s Team`, then the status	**Pass**
2	Window closed, security on	Buzzer silent, LCD `Window: CLOSED`	**Pass**
3	Open the window	Buzzer sounds continuously, LCD `!! INTRUSION !!`	**Pass**
4	Close the window again	Buzzer stops at once, LCD back to `Window: CLOSED`	**Pass**
5	Open and close the window several times quickly	No false beeps or flicker (debounce)	**Pass**
6	Alarm starts	`ALERT:WINDOW_OPEN` printed once on the Serial Monitor	Shown live

4.8 Sending the alert to the recipients (next step)

The Arduino UNO has no Wi-Fi, so it cannot send a message to a phone by itself. Our plan:

1. Add an **ESP8266** Wi-Fi board (ESP-01 or NodeMCU).
2. Arduino **TX (D1)** goes to ESP8266 **RX** through a 1 kΩ / 2 kΩ divider, because the ESP8266 works at 3.3 V.
3. When the ESP8266 receives `ALERT:WINDOW_OPEN`, it sends a **Telegram** message to the team's phones.

The Arduino side is already done: the message is sent once per intrusion.

5. Problems we faced and how we solved them

Problem	Cause	Solution
Compile error: `LiquidCrystal_I2C.h: No such file`	The LCD library was not installed	Installed *LiquidCrystal I2C* (Frank de Brabander) from the Library Manager
Upload error: `cannot open port COM7` / `semaphore timeout`	Wrong COM port selected; the port was busy	Chose the right port in Tools → Port, closed the Serial Monitor and reconnected the USB
LCD showed only black boxes	The Arduino was not talking to the LCD (address / contrast)	Ran an I2C scanner to find the address (`0x27`) and adjusted the contrast screw on the back
LCD always said *Window: CLOSED*	The first LCD program never read the window sensor	Wrote one program that runs the sensor and the LCD together
Buzzer worked alone but not in the combined program	Our buzzer needs a tone signal; a plain HIGH only made it click	Used `tone()` / `noTone()` , which work for both buzzer types
Buzzer kept sounding after the window closed	The first version latched the alarm until a mute button was pressed	Changed the logic so the alarm follows the window: open = sound, closed = silent
Not enough Arduino pins for all five systems	The 4×4 keypad alone needs 8 pins	Planned a PCF8574 I2C module, so the keypad shares the LCD's two I2C wires
First house design needed 3 MDF sheets	Tall walls and a big frame under the lid	Lowered the walls to 150 mm and used 4 small corner locators: now 2 sheets

6. Next steps

Next	Work	Parts still to buy
1	Connect the ARM toggle switch (D7) and set `ALWAYS_ARMED = false`	-
2	Phone alert through ESP8266 + Telegram	ESP8266 (ESP-01 + adapter, or NodeMCU)
3	Outside light with the PIR motion sensor	PIR HC-SR501
4	Indoor lighting (LDR + button + LEDs)	10 kΩ resistor
5	Climate control (DHT11 + fan + knob)	10 kΩ potentiometer, TIP120 + 1N4007 + 1 kΩ, 5 V 2 A adapter
6	Main door (keypad + servo, auto close)	PCF8574 I2C module
7	Move the wiring to a soldered veroboard, write the final report, record the video	Veroboard

Appendix A: Full code (security + LCD)

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

// ================= settings =================
const uint8_t LCD_ADDR        = 0x27;   // the address the scanner found
const uint8_t PIN_REED        = 2;      // MC-38 -> D2 + GND
const uint8_t PIN_BUZZER      = 8;      // buzzer + -> D8, - -> GND
const uint8_t PIN_ARM         = 7;      // toggle middle -> D7, side -> GND
const bool    ALWAYS_ARMED    = true;   // true = no toggle needed; false when the toggle is wired
const uint8_t REED_OPEN_LEVEL = HIGH;   // if the window shows reversed, change to LOW
const unsigned int BUZZ_HZ    = 2500;   // buzzer tone frequency

LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);

bool armed = false, windowOpen = false, alarmOn = false;

/**
 * Read a pin and return its value only after it stays stable for 30 ms.
 * @param pin Arduino pin number
 * @param stable last stable value (updated in place)
 * @param lastRaw last raw reading (updated in place)
 * @param changedAt time of the last raw change (updated in place)
 * @return the stable logic level (HIGH / LOW)
 */
bool readDebounced(uint8_t pin, bool &stable, bool &lastRaw, unsigned long &changedAt) {
  bool raw = digitalRead(pin);
  if (raw != lastRaw) { lastRaw = raw; changedAt = millis(); }
  if (millis() - changedAt > 30) stable = raw;
  return stable;
}

/**
 * Turn the buzzer on (continuous tone) or off. Works for active and passive buzzers.
 * @param on true = sound, false = silent
 * @return void
 */
void buzzer(bool on) {
  static bool isOn = false;
  if (on == isOn) return;
  isOn = on;
  if (on) tone(PIN_BUZZER, BUZZ_HZ); else noTone(PIN_BUZZER);
}

/**
 * Write two lines (padded to 16 chars), redrawing only the lines that changed.
 * @param l1 row 0 text
 * @param l2 row 1 text
 * @return void
 */
void lcdWrite(const char *l1, const char *l2) {
  static char shown[2][17] = { "", "" };
  const char *src[2] = { l1, l2 };
  for (uint8_t r = 0; r < 2; r++) {
    char buf[17];
    snprintf(buf, sizeof(buf), "%-16.16s", src[r]);
    if (strcmp(buf, shown[r]) != 0) {
      lcd.setCursor(0, r);
      lcd.print(buf);
      strcpy(shown[r], buf);
    }
  }
}

/**
 * Read the reed + toggle. Window open while armed = buzzer ON; window closed = buzzer OFF.
 * @return void
 */
```

```
void updateSecurity() {
  static bool rS = HIGH, rL = HIGH, aS = HIGH, aL = HIGH;
  static unsigned long rT = 0, aT = 0;
  static bool lastAlarm = false;

  windowOpen = (readDebounced(PIN_REED, rS, rL, rT) == REED_OPEN_LEVEL);
  armed = ALWAYS_ARMED || (readDebounced(PIN_ARM, aS, aL, aT) == LOW);
  alarmOn = armed && windowOpen;

  buzzer(alarmOn);

  if (alarmOn && !lastAlarm) Serial.println("ALERT:WINDOW_OPEN");   // for the ESP8266 later
  lastAlarm = alarmOn;
}

/**
 * Show the alarm or the normal status on the LCD.
 * @return void
 */
void updateLCD() {
  if (alarmOn) { lcdWrite("!! INTRUSION !!", "Window OPEN"); return; }
  char l1[17], l2[17];
  snprintf(l1, 17, "Security:%s", armed ? "ARMED" : "OFF");
  snprintf(l2, 17, "Window:  %s", windowOpen ? "OPEN" : "CLOSED");
  lcdWrite(l1, l2);
}

/**
 * Configure pins and start the LCD and serial port.
 * @return void
 */
void setup() {
  pinMode(PIN_REED, INPUT_PULLUP);
  pinMode(PIN_ARM, INPUT_PULLUP);
  pinMode(PIN_BUZZER, OUTPUT);
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  lcdWrite("  SMART HOME", "  MCT331s Team");
  delay(1500);
}

/**
 * Main loop: read the sensors, drive the buzzer, update the screen.
 * @return void
 */
void loop() {
  updateSecurity();
  updateLCD();
}
```

Appendix B: Planned pin map (whole house)

Pin	Connected to	System	Status
D2	MC-38 window sensor	Security	**Used**
D3	PIR motion sensor	Security	Planned
D4	DHT11 temperature sensor	Climate	Planned
D5	Fan driver (TIP120)	Climate	Planned
D6	Light override button	Lighting	Planned
D7	ARM toggle switch	Security	Next
D8	Buzzer	Security	**Used**
D9	Door servo SG90	Door	Planned
D10	Exterior light LED	Security	Planned
D11	Indoor LEDs ×3	Lighting	Planned
D12	Alarm mute button	Security	Optional
A0	LDR light sensor	Lighting	Planned
A1	Potentiometer (fan set point)	Climate	Planned
A4 / A5	I2C bus: LCD (0x27) and keypad PCF8574 (0x20)	Display / Door	**LCD used**
D1 (TX)	ESP8266 (phone alerts)	Security	Next

Appendix C: Project files

File / folder	Contents
laser_dxf/variant_B_flat_roof_v2/	The 2 nested cutting sheets and one DXF per part
MCT331s_Smart_Home_Design.html	Interactive 3D model and simulation of the house
wiring/Wiring_Guide.pdf	Step-by-step wiring guide with breadboard diagrams
cad_tools/house_geometry.py	Script that generates all the panels and DXF files